MEMORANDUM

To: Charlie Refvem, Lecturer, Department of Mechanical Engineering, Cal Poly SLO
crefvem@calpoly.edu

From: Quinn Patterson
qpatters@calpoly.edu
Jake Rowen
jhrowen@calpoly.edu

Team: 23

Date: 2/25/26

RE: **Lab 0x05 Memo**

1. Memo
2. Diagrams
3. Attached Files

1. Memo

This weeks lab focused on utilizing our analog line sensor to make Romi able to follow black lines. We wrote our sensor class to initialize the entire array, giving each sensor its own reading value for later calculations. Our sensor task calculates our centroid and has functions to allow us to zero our setpoint and normalize our readings. This way we can have white readings come in close to zero and black readings come close to one. We also wrote a separate line following control task that runs a simple P control to keep us on the line. If the future we want to integrate the two tasks together as they do not need to be separate. The control task uses our centroid to give a change in setpoint, which we send to the motors to adjust the course slightly. This turned out to be very robust and when we performed bump tests the line sensor could be knocked completely off the line and still correct course.

We added 4 new shares (follow, setzero, setnormalize, read), 4 new queues (dLeft, dRight, TimeValuesSensor, DataValuesSensor), and a little bit of UI functionality. In our UI, we created options to read the values of each sensor and the centroid calculation when “r” was pressed. You can zero the sensor with “z” and normalize with “n”. When “l” is pressed, the motors are set to the setpoint given by the user and the line following control is run. The robot will follow the line until the user presses “l” again, which disables the motors. When “l” is pressed for the second time, the user task will print the values from out new “TimeValuesSensor” and “DataValuesCentroid” queues. These queues contain the first 300 times and centroid value readings used by Romi. Each of these 4 keys have their own share that task user actuates to communicate with task motor and task sensor. The two queues are sent by our control loop task to the motor tasks to modify each setpoint respectively. The new queues dLeft and dRight contain the setpoint changes generated by our control loop for steering romi along the line.

2. Diagrams

```
+-------------------------------------------------------------------------------+
| ME 405 Romi Tuning Interface Help Menu                                        |
+---+---------------------------------------------------------------------------+
| k | Enter new gain values                                                     |
| s | Choose a new setpoint                                                     |
| g | Trigger step response and print results                                   |
| z | Trigger zero for line sensor                                              |
| n | Trigger normalize for line sensor                                         |
| r | Trigger read      for line sensor                                         |
| l | Run Line Follower                                                         |
+---+---------------------------------------------------------------------------+
```

Fig 1. Updated UI

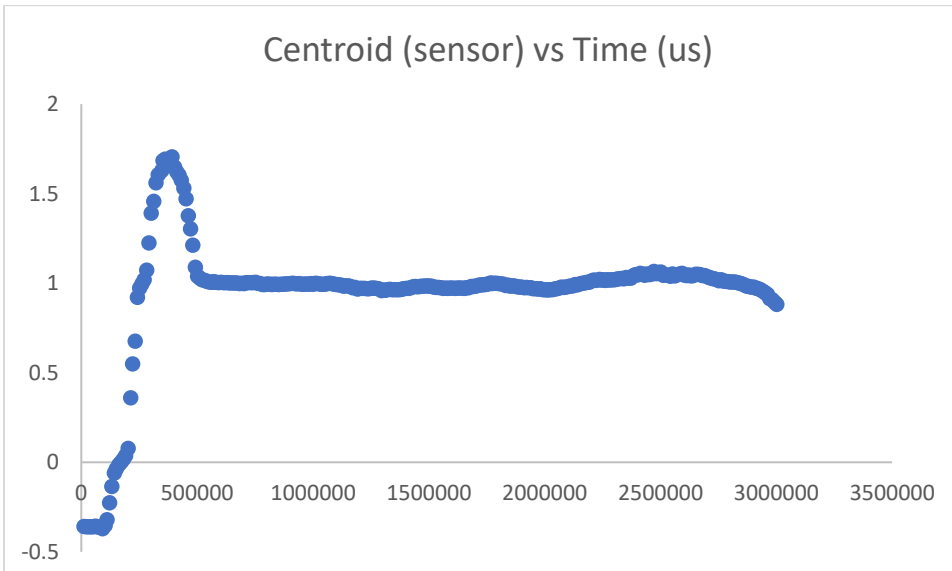

Fig 2. Centroid vs. Time Diagram

Note on Fig 1. Units for time are in microseconds and the units for our centroid are in line sensor positions away from the center sensor. A value of 1 means that a line sensor 1 away from the center sensor is centered on the line.

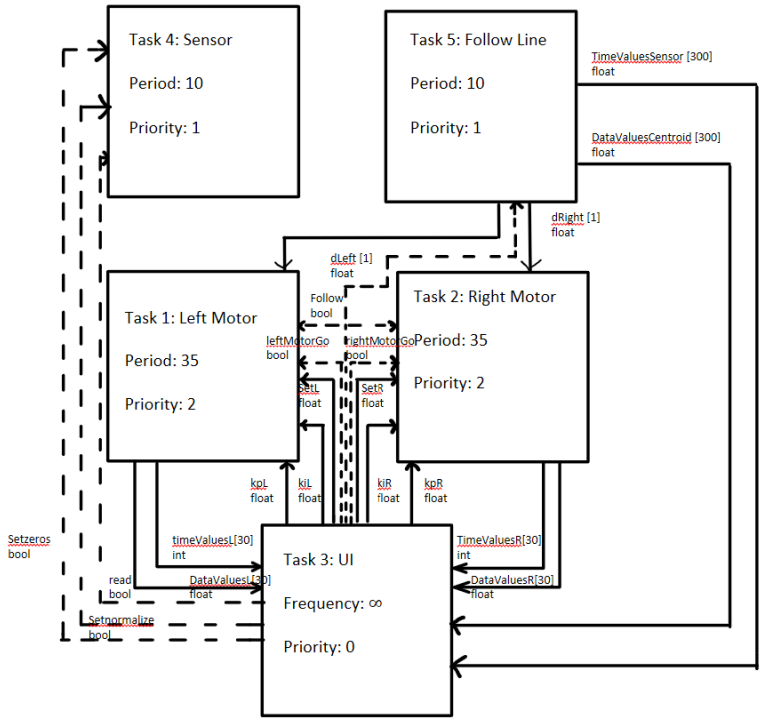

Fig. 3 – Romi system task diagram

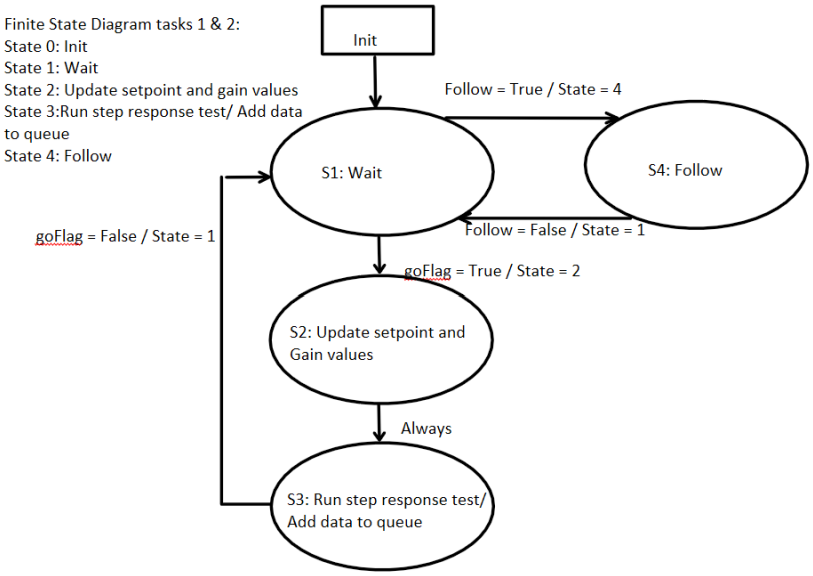

Fig. 4 – Motor Task state transition diagram

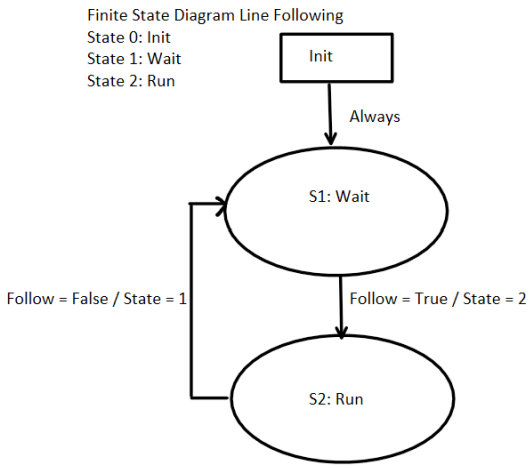

Fig. 5 – Line Follow Task state transition diagram

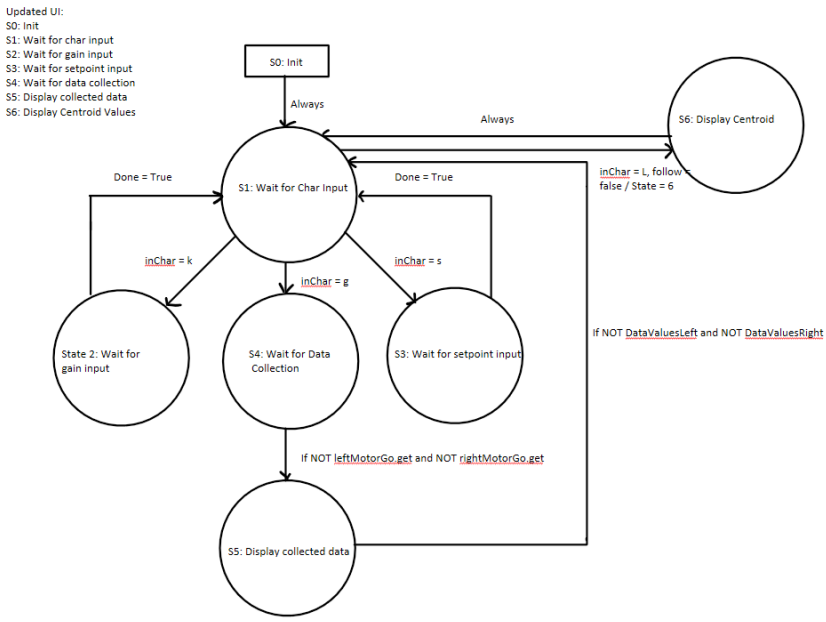

Fig. 6 – UI task state transition diagram

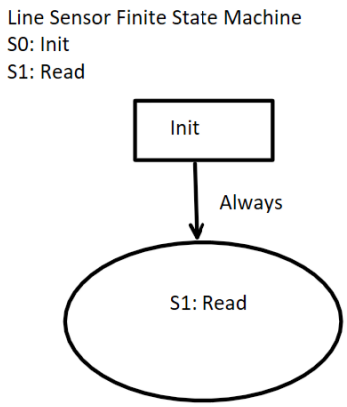

Fig. 7 – Line sensor task state transition diagram

3. Attached Files

Cotask.py
Cqueue.py
Encoder_class.py
Main.py
Motor_class.py
Motor_control.py
Sensor_class.py
Task_follow_line.py
Task_motor.py
Task_sensor.py
Task_share.py
Task_user.py
Line Follow Proof.mov